



浙江大学
ZHEJIANG UNIVERSITY

全局运动规划

《机器人技术与实践》实验报告

课程名称:

机器人技术与实践

小组成员:

金加康 吴必兴 沈学文 钱满亮 赵钰泓 项科深

组 号:

第三组

指导教师:

周春琳

报告日期:

2026 年 1 月

全局运动规划

第三组 金加康 吴必兴 沈学文 钱满亮 赵钰泓 项科深

§ 1 实验任务与分工

1.1 实验目的

1. 理解基于采样的运动规划算法 (RRT、RRT*) 的基本原理与实现流程。
2. 掌握路径规划算法在 ROS 环境下的部署与调试方法。
3. 学习对路径规划算法进行定量分析与改进优化。

1.2 任务要求

1. 在静态障碍物环境下, 生成地图并使用 Nav2 内置全局规划算法进行路径规划
2. 实物机器人通过 Nav2 自带的规划到达设定的目标点。
3. 实现一种全局规划算法 (RRT,RRT*等), 替换 Nav2 自带的全局规划器

1.3 任务分工

- 代码撰写: 金加康、沈学文、吴必兴、赵钰泓、钱满亮、项科深
- 实物调试: 沈学文、金加康、吴必兴、赵钰泓、钱满亮、项科深
- 报告撰写: 沈学文、金加康、吴必兴

§ 2 算法实现

2.1 算法简介

RRT* (Rapidly-exploring Random Tree Star) 是 RRT 算法的改进版本, 由 Karaman 和 Frazzoli 于 2011 年提出。传统 RRT 算法虽然能够快速找到可行路径, 但生成的路径质量往往欠佳。RRT*算法的核心思想是在构建随机树的过程中持续优化路径, 通过父节点重选择 (Choose Parent) 和路径重连 (Rewire) 两个关键机制, 使生成的路径逐步逼近最优解。

相比传统 RRT 算法, RRT*的改进主要体现在以下三个方面:

- 父节点选择: 在添加新节点时, 不直接连接至最近节点, 而是在邻域范围内搜索, 选择使从起点到新节点的累积代价最小的节点作为父节点。
- 路径重连: 新节点加入树后, 检查其邻域内的已有节点, 若这些节点通过新节点重新连接能够获得更小的累积代价, 则更新其父节点关系。
- 渐近最优性: 理论上保证了当采样点数量趋于无穷时, 算法生成的路径将收敛至最优解。

2.2 算法流程

RRT*算法的执行流程如下:

1. 初始化: 将起点设置为随机树的根节点, 配置地图范围和障碍物信息。
2. 随机采样: 在配置空间中随机采样, 以 10% 的概率直接采样目标点。

3. 节点扩展: 从最近节点沿采样方向扩展固定步长 (1.0 m), 生成候选节点。
4. 父节点选择: 在邻域内选择使累积代价最小的节点作为父节点。
5. 路径重连: 检查邻域内已有节点, 若通过新节点能降低代价则更新父节点关系。
6. 迭代终止: 重复上述步骤, 直至找到可行路径或达到最大迭代次数 (1500 次)。

2.3 核心策略

2.3.1 随机采样策略

采用目标偏向采样策略 (Goal-biased Sampling), 以 10% 的概率直接采样目标点, 其余情况在自由空间中均匀随机采样:

$$\text{Sample}() = \begin{cases} x_{\text{goal}} & \text{概率为 } p_{\text{goal}} \\ x_{\text{random}} \in \mathcal{X}_{\text{free}} & \text{概率为 } 1 - p_{\text{goal}} \end{cases} \quad (1)$$

2.3.2 扩展函数 (Steer)

从最近节点沿采样方向扩展固定步长, 扩展函数表达式为:

$$x_{\text{new}} = x_{\text{near}} + \min(\eta, d(x_{\text{near}}, x_{\text{rand}})) \cdot \frac{x_{\text{rand}} - x_{\text{near}}}{\|x_{\text{rand}} - x_{\text{near}}\|} \quad (2)$$

其中 η 为扩展步长, 设置为 1.0 m。

2.3.3 父节点选择 (Choose Parent)

在邻域 $\mathcal{N}_{\text{near}}$ 内搜索, 选择使累积代价最小的节点作为父节点:

$$x_{\text{parent}}^* = \arg \min_{x \in \mathcal{N}_{\text{near}}} \{c(x) + d(x, x_{\text{new}})\} \quad (3)$$

需满足无碰撞约束: $\text{CollisionFree}(\text{Path}(x, x_{\text{new}})) = \text{True}$ 。

2.3.4 重连优化 (Rewire)

新节点加入后, 检查邻域内已有节点, 若满足 $c(x_{\text{new}}) + d(x_{\text{new}}, x_{\text{near}}) < c(x_{\text{near}})$ 且路径无碰撞, 则更新父节点关系。该操作是实现渐近最优性的关键机制。

2.3.5 邻域半径计算

邻域半径随树规模动态调整:

$$r = \min \left(r_0 \cdot \sqrt{\frac{\log(n)}{n}}, 5\eta \right) \quad (4)$$

其中 n 为节点总数, r_0 为初始邻域半径 (8.0 m)。该策略在探索效率与计算复杂度之间取得平衡。

2.4 碰撞检测与路径平滑

2.4.1 碰撞检测

算法采用 KD-Tree 加速最近邻查询。点碰撞检测判定节点到最近障碍物的距离是否大于机器人半径与安全裕度之和 (0.6 m); 线段碰撞检测对路径进行离散化采样 (间隔 0.5 m), 逐点检测碰撞。

2.4.2 路径平滑

采用贪心策略简化路径: 从起点开始, 寻找能直接连接的最远节点, 跳过中间节点, 重复该过程直至终点。实验表明该方法可将节点数减少至原始路径的三分之一左右。

2.5 算法伪代码

RRT*算法的主流程伪代码如下:

```

算法 RRT*路径规划
输入: 起点 x_start, 终点 x_goal, 障碍物集合 Obs, 最大迭代次数 N_max
输出: 路径 Path 或 失败

1: 初始化树 T ← {x_start}
2: for i = 1 to N_max do
3:   x_rand ← Sample()           // 随机采样
4:   x_nearest ← Nearest(T, x_rand) // 查找最近节点
5:   x_new ← Steer(x_nearest, x_rand, η) // 扩展固定步长
6:   if CollisionFree(x_new) then   // 碰撞检测
7:     X_near ← Near(T, x_new, r)   // 查找邻域节点
8:     x_new ← ChooseParent(X_near, x_new) // 选择最优父节点
9:     T ← T ∪ {x_new}             // 加入树
10:    Rewire(T, X_near, x_new)     // 重连优化
11:    if Distance(x_new, x_goal) ≤ η then
12:      Path ← ExtractPath(T, x_new, x_goal)
13:      return SmoothPath(Path)
14:    end if
15:  end if
16: end for
17: return 失败
```

2.6 算法特点分析

RRT*算法在理论层面具有显著优势。算法保证了渐近最优性, 即当迭代次数趋于无穷时路径收敛至最优解, 同时具备概率完备性, 在采样点足够多的情况下必然能找到可行解(若解存在)。算法的环境适应性使其能够处理高维配置空间和复杂障碍物分布, 并且在规划过程中持续改进路径质量, 展现出良好的在线优化能力。

然而, 算法在实际应用中也面临一些挑战。收敛速度是主要瓶颈之一, 达到接近最优效果通常需要数千次迭代, 相应的计算开销也较为可观——时间复杂度为 $O(n \log n)$, 1500 次迭代约需数秒。此外, 算法对参数设置较为敏感, 不同场景需要针对性调整, 而完整随机树结构的存储也带来了一定的内存压力。

2.7 实现优化

针对上述局限性, 本实验采用了多项优化措施。在数据结构层面, 通过 `scipy.spatial.KDTree` 加速障碍物最近邻查询, 将复杂度从 $O(m)$ 降至 $O(\log m)$; 在算法策略上, 采用动态邻域半径 $r = \min\left(r_0 \sqrt{\frac{\log(n)}{n}}, 5\eta\right)$ 平衡探索效率与计算开销, 并通过贪心策略对路径进行平滑处理以去除冗余节点。为加快收敛速度, 实现中引入了目标偏向采样机制, 以 10% 概率直接向目标点采样, 显著提升了算法的实用性。

2.8 参数配置

本实验针对 20×20 m 的 S 型迷宫等复杂场景进行参数调优, 最终确定的参数配置如下表所示:

参数	数值	说明
expand_dis	1.0 m	扩展步长，需在碰撞风险与探索效率间平衡
connect_circle_dist	8.0 m	初始邻域半径，随树规模动态调整
goal_sample_rate	10%	目标采样概率，用于加快收敛速度
max_iter	1500	最大迭代次数，通常可满足规划需求
robot_size	0.5 m	机器人半径，根据实际机器人尺寸设定
safe_dist	0.1 m	安全裕度，提供额外安全边界

§ 3 实物验证

具体实际效果见 video/文件夹下的 task1 task2 文件夹的视频

3.1 Nav2 地图生成

实物机器人采用 TurtleBot 平台，通过 move_base 节点实现路径跟随。下图即为使用手动摇控模式将机器人移动至目标点后，激光雷达构建的栅格地图。

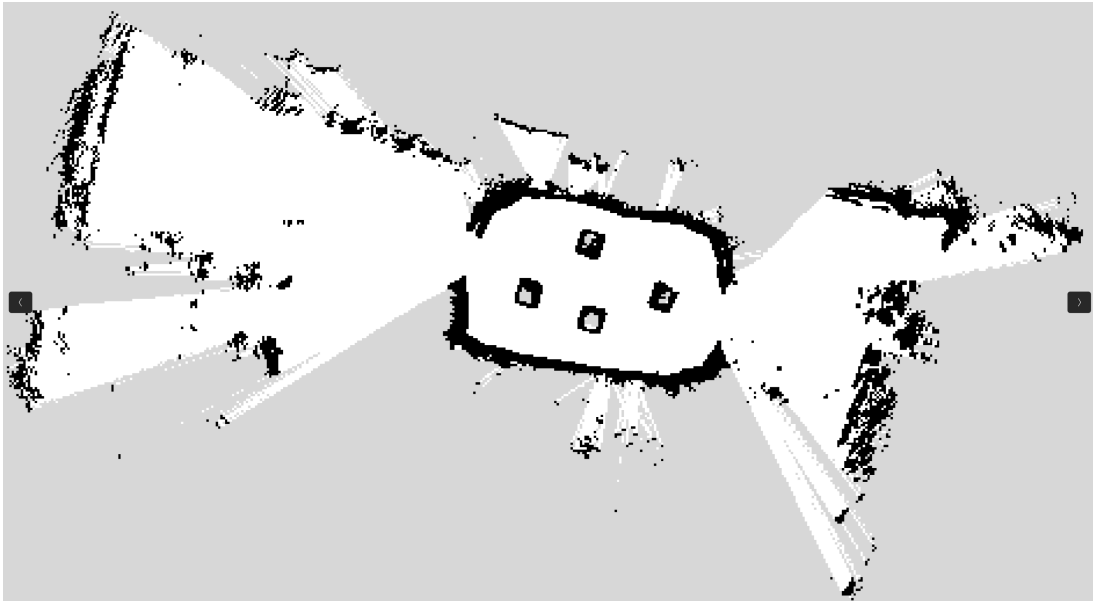


图 1 地图生成结果

3.2 Nav2 原生路径规划

在生成的栅格地图上，使用 Nav2 自带的全局规划器进行路径规划，如下图所示。可以看到，Nav2 规划器成功生成了一条从起点到目标点的路径，机器人能够沿该路径运动避开障碍物。

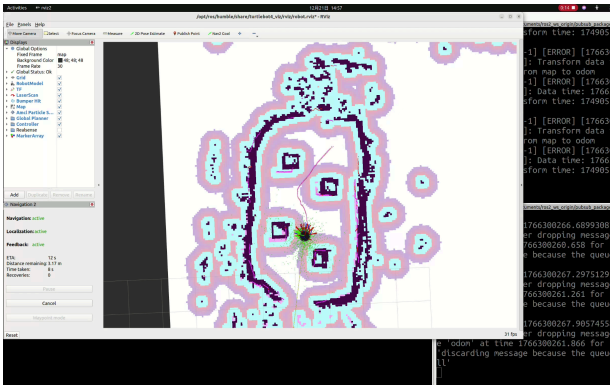


图 3 Nav2 原生路径规划结果



图 4 实物实验到达目标状态

3.3 自主实现 RRT*算法

将实现后的算法部署到 ROS2 环境中，通过 RVIZ 可视化路径规划过程。

我们在移植算法的过程中也出现了一些问题，算法中采取平滑策略，平滑后直线段可能更贴近障碍物边界，存在定位误差或障碍物膨胀不足时，增加碰撞风险，在我们初期的实验中就出现了这样的情况

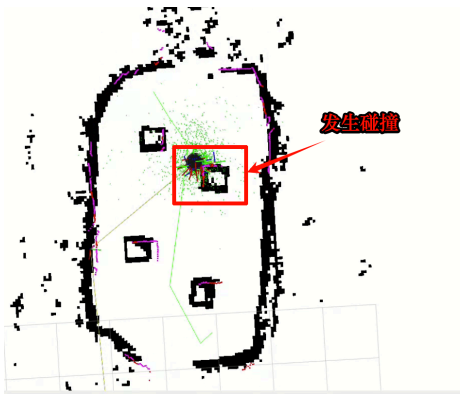


图 5 初期实验中路径与障碍物过近导致碰撞



图 6 碰撞实物图

最后经过对算法的调整，以及对障碍物膨胀的处理，最终实现了较为理想的路径规划效果，如下图所示：

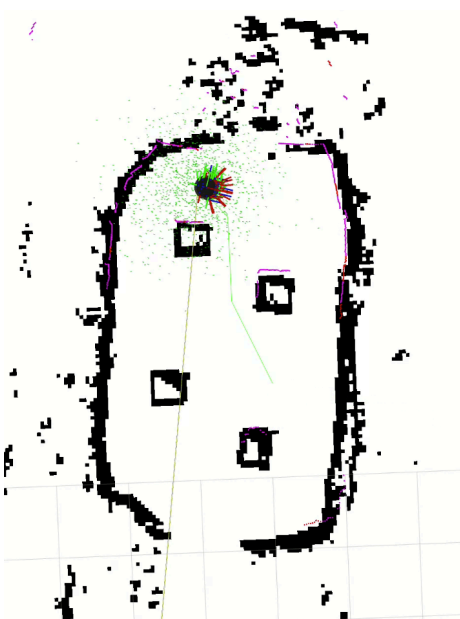


图 7 RVIZ 初始状态

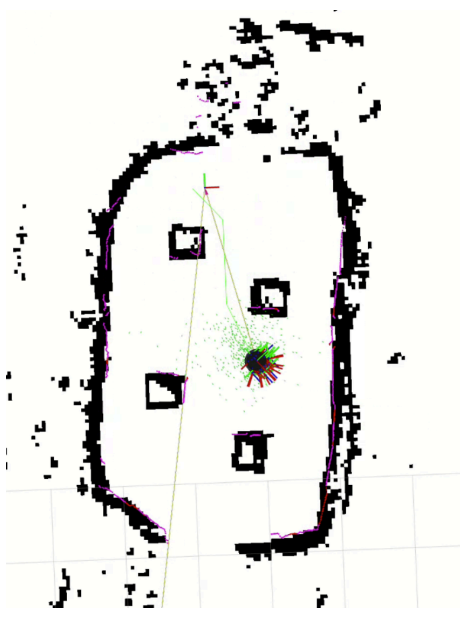


图 8 RVIZ 到达目标状态

图中红色和紫色点云为激光雷达感知的障碍物边界，绿色线条为规划路径。可以看到机器人成功绑定规划路径，绕过障碍物到达目标点。

§ 4 实验总结

本次实验完成了基于 RRT* 的全局运动规划任务，主要成果如下：

1. 完成了地图构建与 Nav2 原生规划器的部署，掌握了 ROS2 环境下路径规划的基本流程。
2. 深入理解了 RRT* 算法的核心机制，包括父节点选择、路径重连等关键策略，并通过 KD-Tree 加速、动态邻域半径等优化措施提升了算法性能。
3. 成功将 RRT* 算法集成到 Nav2 框架中，在实物环境中验证了算法的有效性。
4. 解决了路径平滑策略导致的碰撞问题，通过调整安全裕度和障碍物膨胀参数，实现了安全可靠的路径规划。

通过本次实验，深刻体会到算法从理论到实际应用需要充分考虑环境约束与安全裕度。路径规划不仅要追求路径质量，更要保证系统的鲁棒性和安全性，针对性的参数调优是实现可靠规划的关键。